

ZonDoc / “Zonal Docs” — Complete Design & Reimplementation Specification

Purpose of this document. This is a full, implementation-grade design spec for a desktop application that generates Telangana government “works”/tender agreement documents by mail-merging `.docx` templates against tabular data, plus an in-app high-fidelity docx viewer/editor, a live tender-portal search, and a holiday-calendar dashboard.

It is written so that a *different* coding agent can re-implement the app from scratch with no access to the original source. Where an exact contract matters (shared types, the IPC surface, key algorithms), the spec is precise. Where a choice is free (styling details, minor UX), it says so.

Internal identifiers in the current build are `DocuGen` (appld `com.docugen.app`, `userData` folder `docugen`); the product is branded “Zonal Docs” / “ZonDoc”. Keep the internal id stable if you want to preserve an existing install’s data folder.

0. Coverage — what this document contains

1. Product overview & domain context
 2. Technology stack & exact dependency versions
 3. High-level architecture (3 layers) & rationale
 4. Full project/file structure
 5. Complete shared data model (all TypeScript types, verbatim)
 6. Core (framework-agnostic) modules — every function’s contract & algorithm
 7. The full IPC contract (channel names + typed method signatures)
 8. Electron main process — windows, lifecycle, every IPC handler, the tender-portal bridge, the calendar scraper, persistence & first-run seeding
 9. Preload bridge & renderer API access
 10. Renderer / UI — app shell, tabs, navigation, every component, and end-to-end user workflows
 11. The docx editing engine in depth (the hardest part): DOM↔XML alignment, drag-drop field insertion, formatting-preserving run diff, resilient fill
 12. The design system (visual language, tokens)
 13. Build, packaging, code-signing & distribution
 14. Known issues, gotchas & edge cases
 15. Suggested reimplementation milestones
-

1. Product overview & domain context

What it does. A single-user desktop app for staff who prepare Telangana Government “works” (civil tender / agreement) paperwork. For each *work* (a road/building project), they must produce many

.docx documents (agreement bond, forwarding note, work order, EoT, completion report, etc.) that repeat the same field values (name of work, estimate amount, tender ID, agency name, EMD, dates...). The app stores those field values once (a "Works database"), lets the user upload/prepare **.docx** templates containing **{{Placeholder}}** tokens, and generates one filled document per selected row × template — preserving Word formatting exactly.

Secondary features.

- **In-app docx viewer/editor:** render a template with Word-grade fidelity, drag data fields onto exact caret positions to insert **{{tokens}}**, remove tokens, and inline-edit paragraph text — all writing directly into the **.docx** XML so formatting never degrades.
- **Filled preview & print:** render every filled document to a print-ready window using the same rendering engine; print via the OS.
- **Tender search:** live search of the Telangana e-procurement portal (**tender.telangana.gov.in**) from inside the app.
- **Dashboard:** scrapes the Telangana 2026 holiday calendar and computes tender working-day windows (3-day/7-day) from an anchor date.

Core domain constant — the "Works" schema (28 columns, ordered):

```
Zone, Circle, CNO, Wincode, Name of the work, Amount of estimate, Estimate Amount ECV, Contract Amount, Sanction By, Tender notice, Financial Year, Tender Notice No, Tender notice Date, Tender ID, Tender Percentage, Name of the Agency, Address of the agency, Phone number of the agency, TCV, Intimation Date, EMD 1.5%, EMD 1%, ASD, Reservation, Completion Period, Agmt Date, Technical Sanc No, TS date
```

Template placeholders are matched to these column names by exact string equality (**{{Name of the work}}** ↔ column **Name of the work**).

2. Technology stack & exact versions

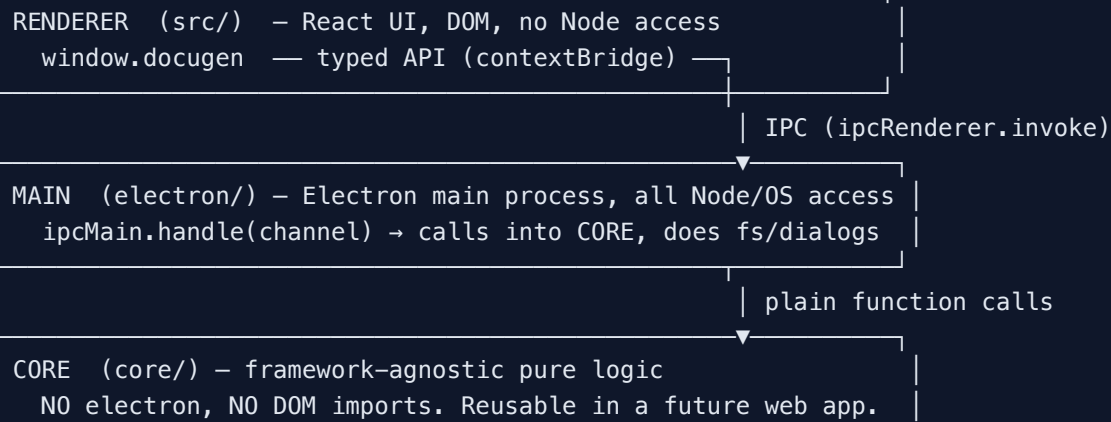
Concern	Choice
Desktop shell	Electron <code>^33.2</code> (built with 33.4.11)
Build/dev tooling	electron-vite <code>^2.3</code> , Vite <code>^5.4</code>
Packaging	electron-builder <code>^25.1.8</code>
Language	TypeScript <code>^5.7</code> , <code>strict: true</code>
UI framework	React <code>^18.3</code> + react-dom , plain function components + hooks (no state library)
Excel parsing	xlsx (SheetJS) <code>0.20.3</code> (from CDN tarball)
docx templating (fill)	docxtemplater <code>^3.50</code> + pizzip <code>^3.1</code>
docx XML editing	pizzip + @xmldom/xmldom (DOMParser/XMLSerializer over <code>word/document.xml</code>)
docx → HTML import	mammoth <code>^1.8</code>
legacy <code>.doc</code> text	word-extractor <code>^1.0</code>
HTML → docx (authoring)	html-to-docx <code>^1.8</code> (used only by the now-dead authoring path)
docx rendering (viewer/preview/thumbnail)	docx-preview <code>^0.4</code> (+ jszip)
docx → PDF (optional)	libreoffice-convert <code>^1.6</code> (requires LibreOffice installed; optional)

Runtime targets: macOS (primary; uses `textutil` for `.doc` → `.docx` and `lp` for printing), Windows (`nsis` target; PowerShell `Start-Process -Verb Print`), Linux (`AppImage` ; `lp`). Node/Electron main-process code uses `fs` , `path` , `os` , `https` , `child_process` .

Module boundaries (tsconfig project references): `tsconfig.node.json` (main+preload+core, Node libs), `tsconfig.web.json` (renderer, DOM libs). Path aliases: `@core/*` → `core/*` , `@/*` → `src/*` . Target ES2022, module ESNext, moduleResolution Bundler.

3. Architecture

Three strictly-separated layers:



Rules that must be preserved:

- `core/` never imports electron or DOM. It only uses Node stdlib + npm libs, and is unit- testable in isolation (there is a vitest suite over it).
- The renderer never touches `fs` /Node directly — everything goes through `window.docugen` , exposed by the preload via `contextBridge` with `contextIsolation: true` , `nodeIntegration: false` , `sandbox: false` (main window).
- The IPC contract (`electron/ipc-contract.ts`) is the single source of truth shared by both preload and main; both import the same `IPC` channel-name map and `DocuGenApi` interface.
- Templates are stored as real `.docx` files under `userData/templates/` ; the app **fills the original bytes in place** with docxtemplater rather than reconstructing documents, which is why formatting is perfectly preserved.

4. Project structure

```
core/                                # framework-agnostic logic (no electron/DOM)
  types.ts                           # ALL shared domain types
  excel.ts                           # xlsx → ExcelTable / SheetGrid (header autodetect)
  sheet.ts                           # raw grid → table; header-row guessing
  merge.ts                           # multi-table merge by row index; collision detect; validate
  templates.ts                       # docxtemplater instance, placeholder extraction, delimiters
  generate.ts                         # fillTemplate (+ resilient fallback), generateDocs, naming re
  naming.ts                           # output filename pattern resolver
  docx-edit.ts                       # XML-level docx editing (insert/remove placeholder, set text)
  export.ts                           # write docx/pdf to disk; docx→pdf via LibreOffice
  preview.ts                         # build standalone print-preview HTML (docx-preview inlined)
  print.ts                           # OS print (lp / PowerShell)
  import.ts                           # any document → editable HTML (mammoth / text / word-extractor)
  authoring.ts                       # editor HTML → docx (html-to-docx) [DEAD PATH]
  calendar.ts                        # parse Telangana holiday calendar HTML; day lookups
  html-to-docx.d.ts, word-extractor.d.ts # ambient type shims

electron/
  ipc-contract.ts                    # IPC channel names + DocuGenApi interface (shared)
  main.ts                            # main process: windows, all ipcMain handlers, tender bridge
  preload.ts                         # contextBridge → window.docugen

src/                                 # renderer (React)
  main.tsx, index.html               # entry + shell (with CSP)
  ipc.ts                             # export const api = window.docugen
  global.d.ts                        # declare window.docugen: DocuGenApi
  worksSchema.ts                     # WORKS_COLUMNS + createWorksTable/applyWorksSchema
  tenderCache.ts                     # in-memory tender search cache + prefetch
  styles.css                         # design system + all component styles
  App.tsx                            # app shell, state, tab routing, persistence
  components/
    Sidebar, Dashboard, GenerateDocsTab, SearchTender, DocPreview, DocThumb,
    ExcelInline, FieldPalette, CollisionPanel, ResultsList, TemplateUpload, Icons
    (dead/unused: GenerateTab, RowPicker, TemplateEditor)

resources/seed-state.json            # bundled first-run works database (no templates)
electron-builder.yml                 # packaging config
electron.vite.config.ts              # electron-vite config (main/preload/renderer)
tsconfig*.json                      # project references
tests/                               # vitest suite over core/
```

5. Shared data model (verbatim)

These types (from `core/types.ts`) are the contract between all layers. Reproduce exactly.

```

export type ExportFormat = 'docx' | 'pdf'

export interface TenderQuery {
  start: number          // 0-based row offset (server pagination)
  length: number          // page size
  type: string            // listing bucket, e.g. 'current'
  search?: string         // server-side keyword (Tender ID / IFB / Name of Work)
}

export interface TenderResult { data: string[][]; total: number } // raw 10-col rows

export interface TemplateFile {
  id: string
  name: string
  path: string            // absolute path under userData/templates
  placeholders: string[]  // unique {{names}} found in the docx
  excelIds?: string[]     // IDs of Excel tables feeding this template ([] / undefined = all)
}

export interface ExcelColumn { name: string; source: string } // source = originating file

export interface ExcelTable {
  id: string
  name: string
  path: string
  headers: string[]
  rows: Record<string, string>[] // row objects keyed by header name
}

export interface Collision { column: string; sources: string[] } // same header in >1 table
export type CollisionResolution = Record<string, string> // column -> winning source

export interface MergedDataset {
  columns: ExcelColumn[]
  rows: Record<string, string>[]
  collisions: Collision[]
}

export interface TemplatePlan { template: TemplateFile; dataset: MergedDataset }

export interface PreviewJob { plans: TemplatePlan[]; namePattern: string; keepUnmatched?: boolean }

export interface GenerateJob {
  plans: TemplatePlan[]
  formats: ExportFormat[]
  outputDir: string
  namePattern: string // tokens: {Template}, {RowIndex}, {ColumnName}
  keepUnmatched?: boolean // keep {{tag}} instead of blanking when no data
  groupBy?: string // optional column -> one subfolder per value
}

export interface GenerateResultItem {
  file: string; template: string; row: number
  format: ExportFormat; ok: boolean; error?: string
}

export interface GenerateProgress { current: number; total: number; file?: string }

```

```

export interface ValidationReport {
  collisions: Collision[]           // unresolved only
  unmatchedPlaceholders: string[]   // placeholders with no matching column
  ok: boolean
}

export interface PersistedState {
  version: number
  tables: ExcelTable[]
  templates: TemplateFile[]
  resolution: CollisionResolution
  formats: ExportFormat[]
  namePattern: string
  keepUnmatched: boolean
  rowSelection: number[] | 'all'    // which rows to output
  wantPrint: boolean                // show print preview after generate
  outputDir: string | null
  templateSel: Record<string, boolean> // which templates are checked
}

```

Additional core-local types:

- `SheetGrid` (`core/sheet.ts`): `{ id, name, path, sheetName, grid: string[][], startRow }` — a raw first-visible-sheet grid before a header row is chosen; `startRow` is the 0-based offset of the grid within the real sheet (Excel row = `startRow + i + 1`).
- `ImportedDoc` (`core/import.ts`): `{ name, path, html, warning? }`.
- `CalendarData` (`core/calendar.ts`): `{ year, months: CalendarMonth[], fetchedAt }`, `CalendarMonth { month, holidays: CalendarHoliday[] }`, `CalendarHoliday { date, name, type: 'public'|'optional' }`.

6. Core modules — function contracts & algorithms

6.1 `core/templates.ts`

- `DELIMITERS = { start: '{{', end: '}}' }`.
- `createDoc(buffer)`: builds a `Docxtemplater` over a `PizZip(buffer)` with `{ delimiters: DELIMITERS, paragraphLoop: true, linebreaks: true }`. (Word can split a placeholder across runs; docxtemplater re-joins runs, so this is more reliable than raw regex on the XML.)
- `extractPlaceholders(buffer): string[]`: try `createDoc(...).getFullText()`; on throw (malformed/partial tags) fall back to `rawDocxText` (strip tags from `word/document.xml`). Then regex `/\{\{s*([^\}]+?)s*\}\}/g`, trim, unique.
- `parseTemplateBuffer(buffer, fileName, filePath)` / `parseTemplateFile(path)` → `TemplateFile` with a generated id `tpl_<base36 time>_<counter>`.

6.2 `core/excel.ts` + `core/sheet.ts`

- `firstVisibleSheetName(wb)`: pick the first sheet whose `Workbook.Sheets[i].Hidden` is falsy (workbooks often hide a config sheet at index 0); fall back to `SheetNames[0]`.

- `readExcelGrid(path): SheetGrid` : read the first visible sheet **cell-by-cell over the decoded !ref range** so blank rows keep their real positions; each cell → `w ?? v` string. Returns the raw grid + `startRow` . (Feeds an optional Header-Row Picker UI.)
- `parseExcelBuffer/parseExcelFile → ExcelTable` : `sheet_to_json({header:1, defval:'', blankrows:false, raw:false})` → grid, then `buildTableFromGrid(grid, guessHeaderRow(grid))` .
- `guessHeaderRow(grid)` : among the first ≤15 rows, choose the row with the **most non-empty cells** (skips a merged one-cell title row, lands on the real header). 0-based.
- `buildTableFromGrid(grid, headerRowIndex, meta)` : header row = chosen row; data rows below; drop fully-empty columns; blank-but-has-data columns named `Column N` ; de-dupe header names with `(2)` , `(3)` ...; skip fully-empty data rows; coerce every cell to string.

6.3 core/merge.ts

- `detectCollisions(tables)` : header name present in ≥2 file **names** → a `Collision` .
- `mergeTables(tables, resolution)` : union columns in first-seen order; each column owned by a source file (colliding columns owned by `resolution[col]` or first-seen); **merge rows by index** — record *n* is built from row *n* of each owning file; `rowCount = max rows across files` ; missing cells → `''` . Returns `MergedDataset` .
- `validate(templates, dataset, resolution)` : `unmatchedPlaceholders` = template placeholders with no matching dataset column; `collisions` = unresolved; `ok` = no unresolved collisions.

6.4 core/naming.ts

- `buildFileName(pattern, templateName, rowIndex, row)` : replace `{Template}` (name minus `.docx`), `{RowIndex}` (1-based), then any `{ColumnName}` with `row[col] ?? ''` ; sanitize `[\\/:*?"<>|]` → `_` ; fall back to `<base>_<n>` if empty.
- `DEFAULT_NAME_PATTERN = '{Template}_{RowIndex}'` . (The app's default is actually `'{Name of the work}-{WIN CODE}'` in seed state.)

6.5 core/generate.ts — the fill engine

- `fillTemplate(templateBuffer, data, keepUnmatched=false): Buffer` :
 - **Try** docxtemplater: `new Docxtemplater(new PizZip(buf), { delimiters, paragraphLoop, linebreaks, nullGetter })` , then `doc.render(data)` , then `getZip().generate({type: 'nodebuffer', compression: 'DEFLATE'})` . `nullGetter` returns `''` for a missing field, or `{{value}}` when `keepUnmatched` .
 - **Catch** (△ the docxtemplater constructor throws eagerly on malformed tags — wrap the *whole* block, not just `render`) : fall back to `fillTemplateResilient` .
 - `fillTemplateResilient(buf, data)` : over each `word/(document|header\d*|footer\d*).xml` , replace tokens matching `/\{\{(?:!(?!\\{) [^\s\S]) *?\}\}/g` ; strip inner XML tags to get the field name; **only** substitute (XML-escaped) when the name is a known `data` key — otherwise leave the braces untouched (so embedded JSON/logs with stray `{{ / }}` never corrupt the doc, and docs that aren't real templates still render). This makes preview and generation crash-proof for **any** input document.
- `generateDocs(job, readTemplate, onProgress?)` : for each plan, read the template bytes once, and for each dataset row produce a `FilledDoc { template, row, data, buffer }` ; report progress `(current, total)` . `readTemplate` is injected (main passes `fs.readFileSync`).

6.6 `core/docx-edit.ts` — XML-level editing (formatting-preserving) — critical

Operates directly on `word/document.xml` via `@xmldom/xmldom`. Namespace `w = http://schemas.openxmlformats.org/wordprocessingml/2006/main`.

- `listParagraphs(buffer): string[]`: text of every `<w:p>` in document order (includes paragraphs inside table cells), each = concat of its `<w:t>` text. **This order is the canonical paragraph index** that the renderer aligns against.
- `resolveParagraphIndex(paragraphs, hint, anchor?): content-addressed resolution`. If no anchor → return `hint`. If `paragraphs[hint]`'s combined text === anchor → `hint` (fast path). Else find all paragraphs whose combined text === anchor and return the one nearest to `hint`. Else `hint`. (Safety net against index drift.)
- `insertPlaceholder(buffer, paragraphIndex, charOffset, column, anchor?): resolve index; build {{column}}`; if the drop offset lands *inside* an existing `{{...}}` token, snap `charOffset` to that token's end (never nest braces); locate the run + local offset by walking run text lengths; insert a new `<w:r>` carrying the placeholder, **copying the neighbour run's `<w:rPr>`** (so formatting matches); split the target run if the offset is mid-run. Empty paragraph → append run. Serialize & rezip.
- `removePlaceholder(buffer, paragraphIndex, column, occurrence=0, anchor?): resolve index; fast path` — a dedicated run whose text is exactly `{{column}}` is removed wholesale; **fallback** — splice the token out of the paragraph's combined run text via `rewriteParagraphRuns`; **resilient fallback** — if the pointed paragraph doesn't contain it (DOM/XML drift in nested tables), search all paragraphs containing the token and operate on the nearest.
- `setParagraphText(buffer, paragraphIndex, newText, anchor?): resolve index; rewrite the paragraph's runs from orig to newText via rewriteParagraphRuns.`
- `rewriteParagraphRuns(xml, runs, orig, newText): longest-common-prefix/suffix diff`. Compute common prefix length `p` and suffix length `s`; the changed span is `[p, orig.length - s]`; the replacement middle is `newText.slice(p, newText.length - s)`. Walk runs by cumulative char range: runs fully outside the changed span are kept byte-for-byte (preserving their `rPr`, including untouched placeholder runs); the run(s) overlapping the span get `keepLeft + middle + keepRight`, with the middle placed once (inheriting the formatting of the run where the edit began). Handles pure insertions at run boundaries and past the last run.

6.7 `core/export.ts`, `core/print.ts`

- `writeOutputs(docxBuffer, {outputDir, baseName, formats}): mkdir -p`; write `.docx`; if `pdf` requested, `docxToPdf` then write; `uniquePath` appends `_2`, `_3` ... to avoid overwrite.
- `docxToPdf(buf): lazy-import libreoffice-convert, promisify(convert)(buf, '.pdf')`. (Fails gracefully if LibreOffice absent — caller records the error per item.)
- `printFile(path): Windows → powershell Start-Process -Verb Print`; else → `lp` (optionally `-d printer`). `printFiles` prints sequentially.

6.8 `core/preview.ts`

- `buildPreviewHtml(docs: {title, base64}[])`: returns a **standalone** HTML page that inlines the browser builds of `jszip` and `docx-preview` (read from `node_modules`, cached), embeds the docs as base64, and renders each with `window.docx.renderAsync(...)` (options: `breakPages`, `renderHeaders/Footers/Footnotes/Endnotes`, `experimental`, `useBase64URL`, etc.). Includes a

sticky non-printing toolbar (Close / Print...), `@media print` rules that hide the toolbar and drop shadows/margins. This is the exact-fidelity print preview.

6.9 `core/import.ts` (used by the “New from document” path)

- `importDocumentBuffer(buffer, fileName, filePath) : .docx → mammoth.convertToHtml ; .html/.htm → as-is; text-like (.txt/.md/.markdown/.csv/.log/.text) → escaped paragraph HTML (blank-line paragraphs,
), guarded by a binary-content sniff; .doc → word-extractor body text (then mammoth, then “unsupported” message); unknown binary → try word-extractor else a clear “convert to .docx/.txt” message (never dumps raw bytes).`

6.10 `core/calendar.ts`

- `parseCalendarHtml(html, year='2026') : find <table>...</table> blocks; per table strip HTML comments and match rows <th class="fix|opt">MMM DD</th><td>Name</td> ( fix =public, opt =optional); decode entities; derive month name from the first holiday's 3-letter abbr.`
- Helpers: `holidayDay(label)` (first number in the label), `holidaysByDay(data, monthIndex)` (day→{name,type}, public wins ties), `MONTH_NAMES`.

7. IPC contract (`electron/ipc-contract.ts`)

Channel-name map (`IPC`) and the `DocuGenApi` interface below are shared by preload + main. Reproduce channel strings exactly.

```
export const IPC = {
  pickTemplates: 'dialog:pickTemplates', pickExcels: 'dialog:pickExcels',
  pickExcelGrids: 'dialog:pickExcelGrids', pickOutputDir: 'dialog:pickOutputDir',
  pickDocuments: 'dialog:pickDocuments', saveTemplate: 'template:save',
  rescanTemplate: 'template:rescan', merge: 'data:merge', validate: 'data:validate',
  generate: 'generate:run', generateProgress: 'generate:progress',
  printPreview: 'print:preview', print: 'print:files', openPath: 'shell:openPath',
  revealItem: 'shell:revealItem', defaultDir: 'shell:defaultDir',
  readDocBytes: 'doc:readBytes', previewFilled: 'doc:previewFilled',
  insertField: 'doc:insertField', removeField: 'doc:removeField',
  saveParagraph: 'doc:saveParagraph', dbgParagraphs: 'doc:dbgParagraphs',
  dbg: 'doc:dbg', paragraphTexts: 'doc:paragraphTexts', fetchCalendar: 'calendar:fetch',
  searchTenders: 'tenders:search', loadState: 'state:load', saveState: 'state:save'
} as const
```

```

export interface DocuGenApi {
  pickTemplates(): Promise<TemplateFile[]>
  pickExcels(): Promise<ExcelTable[]>
  pickExcelGrids(): Promise<SheetGrid[]>
  pickOutputDir(): Promise<string | null>
  pickDocuments(): Promise<ImportedDoc[]>
  saveTemplate(name: string, html: string): Promise<TemplateFile>
  rescanTemplate(path: string): Promise<string[]>
  merge(tables: ExcelTable[], resolution: CollisionResolution): Promise<MergedDataset>
  validate(templates, dataset, resolution): Promise<ValidationReport>
  generate(job: GenerateJob): Promise<GenerateResultItem[]>
  onProgress(cb: (p: GenerateProgress) => void): () => void // subscribe; returns unsubscribe
  printPreview(job: PreviewJob): Promise<{ count: number }>
  print(filePaths: string[]): Promise<void>
  openPath(target: string): Promise<void> // http(s)→openExternal else openFile
  revealItem(target: string): Promise<void>
  defaultDir(): Promise<string> // app downloads dir
  readDocBytes(path: string): Promise<string> // base64 of the .docx
  previewFilled(path: string, data: Record<string, string>): Promise<string> // base64 filled
  insertField(path, paragraphIndex, charOffset, column, anchor?): Promise<{base64, placeholders}>
  removeField(path, paragraphIndex, column, occurrence, anchor?): Promise<{base64, placeholders}>
  saveParagraph(path, paragraphIndex, text, anchor?): Promise<{base64, placeholders}>
  paragraphTexts(path: string): Promise<string[]> // XML paragraph texts (alignment)
  dbgParagraphs(path, domTexts): Promise<void> // diagnostics only
  dbg(tag, data): Promise<void> // diagnostics only
  fetchCalendar(force?: boolean): Promise<CalendarData>
  searchTenders(query: TenderQuery): Promise<TenderResult>
  loadState(): Promise<PersistedState | null>
  saveState(state: PersistedState): Promise<void>
}

```

`dbg` / `dbgParagraphs` / `paragraphTexts` were added for debugging the editor; `paragraphTexts` is **functionally required** (alignment), the other two are diagnostic and can be dropped in a clean reimplementation.

Preload (`electron/preload.ts`) simply maps each method to `ipcRenderer.invoke(IPC.x, ...args)` and exposes it via `contextBridge.exposeInMainWorld('docugen', api)`. `onProgress` uses `ipcRenderer.on( IPC.generateProgress, listener )` and returns a disposer.

8. Electron main process (`electron/main.ts`)

Window: `BrowserWindow` 1200×820 (min 900×640), `title: 'Agreement Desk'`, `backgroundColor: '#fff'`, `webPreferences { preload, sandbox:false, contextIsolation:true, nodeIntegration:false }`. Loads `ELECTRON_RENDERER_URL` in dev, else `../renderer/index.html`. Standard macOS lifecycle (`activate` recreates window; `window-all-closed` quits except on darwin). `registerHandlers()` runs on `whenReady`.

Handler summary (all `ipcMain.handle`):

- **pickTemplates**: open-file dialog (docx/doc/rtf/odt/txt/html..., multi). Copy each pick into `userData/templates/` with a unique name; non-`.docx` converted via macOS `textutil -convert docx`; parse each into a `TemplateFile`. Warn (message box) on conversion failures.
- **pickExcels**: pick xlsx/xls → `parseExcelFile` each (header autodetect).
- **pickExcelGrids**: pick xlsx/xls → `readExcelGrid` each (raw grid for a header picker).
- **pickOutputDir**: pick a directory (`createDirectory`).
- **pickDocuments**: pick docs → `importDocumentFile` each (`.doc` first converted via `textutil` to a temp `.docx`, then mammoth, keeping the original path/name for display).
- **saveTemplate(name, html)**: `htmlToTemplateDocx(html)` → write `userData/templates/<safe>.docx` (unique) → return `TemplateFile`. (Authoring path; currently unreachable.)
- **rescanTemplate(path)**: re-read a docx → `extractPlaceholders`.
- **merge / validate**: delegate to `core/merge`.
- **generate(job)**: `generateDocs(job, fs.readFileSync, progress→sender.send(generateProgress))`; for each filled doc build `baseName` via `buildFileName`, apply optional `groupBy` subfolder (sanitized), `writeOutputs`; collect `GenerateResultItem`s (ok / error per file).
- **print(filePaths)**: `printFiles`.
- **printPreview(job)**: `generateDocs` (no write) → `buildPreviewHtml(docs)` → write temp HTML → open a dedicated 960×900 `BrowserWindow` (sandboxed) on that file; clean up temp on close and refocus the main window. Returns `{count}`.
- **openPath / revealItem / defaultDir**: `shell.openExternal|openPath`, `shell.showItemInFolder`, `app.getPath('downloads')`.
- **readDocBytes(path)**: base64 of the file.
- **previewFilled(path, data)**: `fillTemplate(read(path), data, false)` → base64.
- **insertField / removeField / saveParagraph**: read the docx, call the matching `core/docx-edit` function (with `anchor`), **write the file back**, return `{ base64, placeholders: extractPlaceholders(updated) }`. (These persist edits immediately.)
- **paragraphTexts(path)**: `listParagraphs(read(path))`.
- **fetchCalendar(force?)**: serve `userData/calendar-cache.json` unless `force`; else `https.get` the Telangana 2026 calendar page (browser-like headers, 15s timeout) → `parseCalendarHtml` → cache → return.
- **searchTenders(query)**: `fetchTenders(query)` (see bridge) → parse JSON → `{ data: aaData, total: iTotatDisplayRecords ?? iTotatRecords ?? -1 }`.
- **loadState / saveState**: read/write `userData/state.json`; **first-run seeding** — if `state.json` is missing, load the bundled `seed-state.json` from `process.resourcesPath` (or dev fallbacks under `app.getAppPath()`), returning the works database with no templates. `saveState` writes JSON (best-effort).

Tender portal bridge (the tricky integration). The portal (`https://tender.telangana.gov.in`) needs a real browser session (JSESSIONID) and blocks raw HTTP clients with a WAF. Solution: host a **hidden BrowserWindow** on the site's own origin and run the JSON `fetch` inside it (same-origin, real cookies, no CORS):

- `makeTenderWindow()`: hidden window on a **fresh in-memory partition** `tender-<Date.now()>` (never persistent — an expired persistent JSESSIONID redirects to a timeout page forever).
- `seatTenderSession(win)`: load site root (`/`) then `TenderDetailsHome.html` to establish the listing context.

- `buildTenderUrl(query)` : reconstruct the **full** DataTables querystring the portal's own page sends (dozens of `hdn*` params + `iColumns=10` , `sColumns` , per-column `mDataProp_i` / `bSortable_i` , sort col 5 desc, `iDisplayStart/Length` , cache-buster `_`); keyword goes in `nTenderID` ; listing bucket in `hdnType` . A partial param set returns HTML.
- `runTenderFetch(win, url)` : `webContents.executeJavaScript` a `fetch(url, {headers: {x-requested-with:XMLHttpRequest, accept: json}, credentials:'include'})` → text.
- `fetchTenders(query)` : fast path reuse a live window; else recreate + seat + retry (≤ 2); accept only responses that look like JSON (`{` -prefixed).

9. Renderer / UI

9.1 App shell (`src/App.tsx`)

Single React tree, all state via `useState` / `useRef` (no external store).

Top-level state: `tab` , `calendar` ; workspace `templates` , `tables` , `resolution` ; generate settings `formats` , `outputDir` , `namePattern` , `keepUnmatched` , `rowSelection` , `wantPrint` , `templateSel` ; runtime `generating` , `progress` , `results` ; editor/preview `editorDocs` , `editorIndex` , `previewTemplate` .

Startup: `api.loadState()` → hydrate all persisted fields (if state is empty, ensure a Works table exists via `createWorksTable` / `applyWorksSchema`); `prefetchTenders()` warms the tender cache. **Persistence:** a debounced effect calls `api.saveState({...})` whenever persisted state changes.

Derived: `dataset = mergeTables(tables, resolution)` (client-side, reusing core logic); `datasetForTemplate(t)` selects a template's assigned Excel tables (via `excelIds` , else all); `chosenTemplates` / `effectivePlans` from `templateSel` .

Tabs (via `<Sidebar>`): `dashboard` , `data` , `generate` , `templates` , `search` .

Actions: `generate()` → `api.generate(job)` (+ optional `api.printPreview` when `wantPrint`); template upload → `api.pickTemplates()` ; "New from document" → `api.pickTemplates()` then open first in `<DocPreview>` ; preview/edit an existing template → `<DocPreview template=... tables=... onEdited=.../>` .

9.2 Components

Component	Role	Key IPC
Sidebar	Nav rail (264px, glassy) + status badges (table/template/output counts, unresolved collisions, ready/generating)	—
Dashboard	2026 holiday calendar grid + anchor-date working-day (3/7-day) tender-window calculator; month paging	<code>fetchCalendar</code> , <code>openPath</code>
ExcelInline	Spreadsheet-like editor for the Works table: rename columns, edit cells, add/delete rows & columns, in-table search; auto-commits	—
CollisionPanel	Resolve duplicate column names across files before generating	—
TemplateUpload	Manage template list: upload, “New from document”, open in Word/viewer, show placeholders + linked DBs, delete w/ confirm	(parent callbacks)
GenerateDocsTab	Simplified generate screen: pick templates, filter works by a column value, preview a template, run docx/pdf/print	(parent <code>onGenerate</code>)
DocPreview	High-fidelity docx viewer/editor + filled preview; drag-drop fields, inline edit, remove pills (see §11)	<code>readDocBytes</code> , <code>previewFilled</code> , <code>paragraphTexts</code> , <code>insertField</code> , <code>removeField</code> , <code>saveParagraph</code> , <code>openPath</code>
DocThumb	First-page thumbnail for template tiles (docx-preview), icon fallback	<code>readDocBytes</code>
FieldPalette	Draggable field chips grouped by Excel source, searchable; drag/click inserts <code>{{field}}</code>	—
ResultsList	Post-generation results: open files, print PDFs, show failures, clear	<code>print</code> , <code>openPath</code>
SearchTender	Live tender search: debounced keyword, paging, refresh, formatting, empty/error states	<code>searchTenders</code> (via cache)
Icons	Shared inline SVG set	—

Dead/unused (do not reimplement): `GenerateTab.tsx` , `RowPicker.tsx` , `TemplateEditor.tsx` (+ the `saveTemplate` / `core/authoring.ts` path it drives). Keep only if you want an HTML-authoring editor; the primary flow edits real docx via `DocPreview` .

9.3 `tenderCache.ts`

In-memory promise cache keyed by query; `DEFAULT_TENDER_QUERY` ; `fetchTenders(q, force?)` dedupes concurrent identical queries; `prefetchTenders()` warms the default listing at startup so the Search tab is instant.

10. The docx editing engine (DocPreview + docx-edit) — deep dive

This is the app's hardest, most valuable subsystem. Goal: a Word-grade WYSIWYG editor where the user drags data fields to exact caret positions and inline-edits text, with **zero** formatting loss, writing straight into the `.docx` XML.

Rendering. `DocPreview` renders the docx bytes with `docx-preview`'s `renderAsync(bytes, container, undefined, opts)` into a scrollable "page" surface. It then post-processes the DOM: wraps every `{{placeholder}}` in an atomic, non-editable "pill" span with a `×` remove button (the pill's text stays `{{col}}` so it round-trips), and makes the body `contentEditable`.

The core problem — DOM $\leftrightarrow$ XML paragraph drift. `docx-preview` emits *more* `<p>` elements than the document has `<w:p>` (synthetic empty paragraphs for page breaks/spacing — e.g. a real doc with 3542 `<w:p>` renders ~3605 `<p>`). So "the Nth rendered paragraph" $\neq$ "the Nth XML paragraph", and the drift grows down the document. Any edit keyed on the DOM ordinal hits the wrong XML paragraph.

The fix — `alignDomToXml(domTexts, xmlTexts)` (general, no hardcoding):

- Fetch the true XML paragraph texts via `api.paragraphTexts(path)` (= `listParagraphs`).
- Collect the rendered paragraphs' texts (`domTexts`).
- Greedily align: normalize both by stripping *all* whitespace (`/\s+/g` $\rightarrow$ `''`) to tolerate tab/line-break rendering differences. Walk `dom` with an XML cursor `j`: if `dom[i]===xml[j]` map `i` $\rightarrow$ `j`, `j++`; else if `dom[i]` is non-empty, scan forward in `xml` for a match (resync on distinctive content); else (synthetic empty) leave unmatched. Finally, unmatched entries inherit the previous real XML index.
- Label each rendered `<p>` with `data-pidx = its true XML index`. Return `{map, consumed}`; `consumed === xmlTexts.length` $\Rightarrow$ perfect alignment.
- After alignment, snapshot each paragraph's on-disk text into `baseText: Map<pidx, string>` (the **content anchor** for edits) and clear any stale pending save.

Editing operations (all keyed by `data-pidx` + `anchor`):

- **Inline text edit** $\rightarrow$ debounced `flushSave`: read the paragraph's **live DOM text**, look up its `anchor` from `baseText`, call `api.saveParagraph(path, pidx, text, anchor)`; on success update `baseText[pidx]`. Using the live text + anchor (not a captured value) plus the debounce is what stopped the "old content came back" bug.
- **Drag-drop field insert** $\rightarrow$ `onDrop` uses a 3-tier caret resolver: (1) `document.caretRangeFromPoint(x,y)` $\rightarrow$ the enclosing `p[data-pidx]` + char offset measured via a Range; (2) fall back to the paragraph element under the point; (3) nearest paragraph in the table cell / by geometry. Force-save the target paragraph's current text first, then `api.insertField(path, pidx, offset, column, anchor)`. This lets fields land **mid-text at the exact cursor**, not just at paragraph start.
- **Remove pill** (`×`) $\rightarrow$ `api.removeField(path, pidx, column, occurrence, anchor)`.

All three main-process handlers write the updated bytes to disk and return `{ base64, placeholders }`, so `DocPreview` re-renders from the fresh bytes and refreshes the field list. Because the core edit functions also take `anchor` and run `resolveParagraphIndex`, even a wrong hint is corrected by matching paragraph text.

Why it preserves formatting: edits never round-trip through HTML. Inserts copy a neighbour run's `<w:rPr>`; text rewrites use the prefix/suffix run-diff that leaves untouched runs (and any placeholder runs) byte-for-byte identical.

Known residual issue: on a document that has been heavily edited in-session, alignment can report `ok:false` (e.g. `consumed 30/32`) when a paragraph's rendered text no longer matches any XML text; the nearest-inherit fallback + content anchoring keep behavior reasonable, but a more robust alignment (or a post-alignment verification/fallback) would be worth hardening. (A full LCS/DP alignment was rejected for cost on 3600×3542 paragraphs; the greedy scan is the chosen tradeoff.)

11. Design system (`src/styles.css`)

Visual language: soft indigo/violet brand on white cards over a pale blue/purple gradient; generous rounded corners, subtle shadows, a glassy blurred sidebar. Implement as CSS custom properties on `:root`:

- **Palette:** `--indigo-50/100/500/600/700`, `--violet-500`.
- **Surfaces:** `--bg-grad-a/b`, `--surface`, `--surface-2`.
- **Lines/text:** `--border`, `--border-strong`, `--text`, `--text-2`, `--muted`.
- **Status:** `--ok` / `--ok-bg`, `--warn` / `--warn-bg`, `--danger` / `--danger-bg`.
- **Shape/motion/type:** `--radius`, `--radius-sm`, `--radius-pill`, `--shadow-*`, `--ease`, `--font` (system UI stack). Structural classes: `.shell` (flex frame), `.sidebar` (sticky 264px, blurred), `.workspace` (scroll pane), `.page` / `.card` / `.page-head` / `.head-ic` / `.count-pill`; buttons `.primary`, `.ghost`, `.danger-ghost`, `.btn-lg`; file rows/tags/badges; editor overlay/modal/toolbar + `.doc-editor` / `.ph` (pill); palette chips + searchable groups; sheet/grid editor (`.sheet-inline`, `.rownum ...`); tender/calendar (`.tender-*`, `.cal-*`); notices (`.notice.warn|error`, `.status-dot.ready|warn`). Font stack: `-apple-system`, `BlinkMacSystemFont`, `'Segoe UI'`, `system-ui`, `sans-serif`. Renderer CSP (index.html): `default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self'; img-src 'self' data:.`

12. Build, packaging & distribution

Scripts (`package.json`): `dev` (`electron-vite dev`), `build` (`electron-vite build`), `start` (`electron-vite preview`), `test` (`vitest run`), `typecheck` (`tsc --noEmit` over both project references), `package` (`electron-vite build && electron-builder`), `package:dir` (`... --dir`, unpacked app only).

electron-vite config: three roots — `main` (`electron/main.ts`, `externalizeDepsPlugin`, alias `@core`), `preload` (`electron/preload.ts`), `renderer` (root `src/`, React plugin, aliases `@core`, `@`). Output → `out/{main,preload,renderer}`.

electron-builder.yml: `appId: com.docugen.app`, `productName: DocuGen`, output `release/`; `files: out/**/*`, `package.json`; `extraResources: resources/seed-state.json → seed-state.json`; `mac.target: [dmg, zip]` (category productivity), `win.target: nsis`, `linux.target: AppImage`.

Signing / Gatekeeper (macOS): the app is not notarized. electron-builder with

`CSC_IDENTITY_AUTO_DISCOVERY=false` produces only a *linker* ad-hoc signature (resources not sealed) → recipients see “**damaged / can't be opened**”. Mitigation used: **deep ad-hoc sign** (`codesign --force --deep --sign - --identifier com.docugen.app <App>.app` , which seals resources and passes `codesign --verify --deep --strict`), then build the DMG from the signed app (e.g. `hdiutil create -volname "Zonal Docs" -srcfolder <stage> -format UDZO`). Recipients still must clear quarantine once: `xattr -cr /Applications/DocuGen.app` or System Settings → Privacy & Security → **Open Anyway**. To remove that step entirely requires a paid Apple Developer ID + notarization. Builds are currently arm64-only; use electron-builder `--universal` /arch config for Intel Macs.

First-run data: `resources/seed-state.json` is a `PersistedState` with the Works database rows but `templates: []` , `outputDir: ''` , `templateSel: {}` , `rowSelection: 'all'` . Fresh installs load it via the `loadState` seeding path. Recipients upload their own templates.

13. Known issues, gotchas & non-obvious decisions

- **docxtemplater throws in its *constructor*** on malformed tags, not just `render()` — wrap the whole fill in try/catch (see §6.5). The resilient fallback must only replace *known* fields and leave other braces alone.
- **docx-preview emits extra empty paragraphs** — never assume DOM paragraph N == XML `<w:p>` N. Always align via `paragraphTexts` + `alignDomToXml` and key edits on `data-pidx` .
- **Whitespace-normalized matching** in alignment can, in theory, make two distinct paragraphs compare equal; greedy order mitigates it. Heavy in-session edits can yield `ok:false` .
- **Merge is by row index across files**, not by a join key — record N uses row N of every file. This is intentional for this workflow.
- **Header autodetection** picks the row with the most non-empty cells within the first 15 rows (skips merged title banners).
- **macOS-only helpers:** `.doc` → `.docx` uses `textutil` ; printing uses `lp` . Provide equivalents on Windows/Linux (PowerShell print is already handled; `.doc` import falls back to word-extractor text).
- **PDF export needs LibreOffice** installed; otherwise per-item PDF errors are recorded.
- **Tender portal** is fragile by nature (full DataTables param set + live session in a hidden window). Expect to adjust params if the portal changes.
- **Diagnostic IPC** (`dbg` , `dbgParagraphs`) and a `userData/debug.log` writer exist for the editor; drop them in a clean build (keep `paragraphTexts`).
- **Bundle size:** the DMG is ~100–120 MB (Electron). Distribute via cloud storage.

14. Suggested reimplementation milestones

1. **Scaffold:** electron-vite + React + TS, 3-layer split, IPC contract, `window.docugen` .
2. **Data model & core:** port `types.ts` , `sheet/excel` , `merge` , `naming` , `templates` , `generate` (with resilient fallback); add a vitest suite (header detection, merge, fill).
3. **Works DB UI:** `ExcelInline` over `worksSchema` ; persistence (`loadState/saveState` + seeding).
4. **Templates:** upload/convert (`pickTemplates` + `textutil`), `TemplateUpload` , thumbnails.
5. **Generate:** `generateDocs` + `writeOutputs` , `GenerateDocsTab` , `ResultsList` , progress.

6. **Preview/print:** `buildPreviewHtml` + `printPreview` window; `printFiles` .
7. **docx editor (hardest):** `docx-edit` (insert/remove/setText + run-diff + anchor), `DocPreview` with `alignDomToXml` , pills, 3-tier drop resolver, debounced save.
8. **Tender search:** hidden-window bridge + `SearchTender` + cache.
9. **Dashboard:** calendar scrape/parse + working-day calculator.
10. **Design system, packaging, signing, distribution.**

Ship criteria: generation preserves Word formatting exactly; drag-drop lands fields at the exact caret in the correct paragraph across arbitrary documents; preview/print match Word; first run starts with the seeded Works DB.

